

Backlog:

- Add more sprites/layering into the boss room + logic to bring the spikes into the boss room after entering
- Adding updated sprites for the knight -> as some of the sprites have a blurred effect.

Documentation - Did they say what they were going to do?:

Yes, there were very clear and outlined roles. I can show them here:

Task Breakdown & Assignments

1. Carry-Overs (Sri + Thomas + Sudhish — close out Week 1)

Priority 1. None of the Week 2 features start cleanly until these close. Sri pairs with Thomas on the first dungeon; Sudhish is pre-authorized to pick up boss sprites if Thomas is blocked.

Task	Description	Assignee	Branch
HUD health fix	Debug gameplay-vs-pause regression on health display; reconcile with merged HUD layout	Sri	`feature/hud-health-fix`
Ability UI finish	Complete ability selection UI and wire into HUD/inventory flow	Sri	`feature/ability-ui`
Downslash fix (v2)	Re-fix downslash attack hitbox/animation interaction; regression-test against S4 bug	Sri	`feature/downslash-fix-v2`
First dungeon completion	Finish remaining rooms, connections, and enemy/item placement so dungeon is playable end-to-end	Thomas	`feature/first-dungeon-complete`
First dungeon assist	Pair with Thomas — build out secondary rooms and transitions	Sri	`feature/first-dungeon-assist`

2. Boss Fight (Zach — primary, Sudhish — assist)

Zach owns attack patterns, phases, and room-entry trigger. Sudhish handles integration with collision/physics and the IBreakable/IInteractable work needed for boss-room interactables.

Task	Description	Assignee	Branch
Boss behavior / AI	Implement attack patterns, phase transitions, movement state machine	Zach	`feature/boss-behavior`
Boss room integration	Trigger fight on room entry, implement win/lose conditions, cleanup on victory	Zach	`feature/boss-room`
Boss physics integration	Wire boss hitboxes/hurtboxes into collision system; damage exchange	Sudhish	`feature/boss-integration`

3. Level Layers (Niko + Thomas — visual, Sudhish — collision)

Visual and collision layers ship in parallel. Visual layers are parallax/foreground/background; collision layers are logical layers that determine what collides with what.

Task	Description	Assignee	Branch
Visual layer system	Parallax + foreground + background rendering pipeline	Niko	`feature/layers-visual`
Visual layer assets	Supply and wire background/foreground art for first dungeon	Thomas	`feature/layers-visual-assets`
Collision layer system	Logical collision layers (player, enemy, environment, hazard, projectile)	Sudhish	`feature/layers-collision`

4. Second Dungeon (Thomas — primary, Zach — XML assist)

Second dungeon must be visually distinct from the first. Thomas leads layout and art; Zach assists on XML wiring if Thomas falls behind after Week 1.

Task	Description	Assignee	Branch
Second dungeon layout	Design and build rooms, connections, and enemy/item placement for dungeon 2	Thomas	`feature/second-dungeon-layout`
Second dungeon custom sprite	New tileset and sprite theme distinct from the first dungeon	Thomas	`feature/second-dungeon-sprites`
Second dungeon XML	Author level XML for the new dungeon; integrate with room transitions	Zach	`feature/second-dungeon-xml`

5. Polish & Interfaces (Sudhish + Zach — primary, All — magic numbers)

Task	Description	Assignee	Branch
IBreakable interface	Formalize and apply to walls/grass/breakables	Sudhish	`feature/ibreakable`
IInteractable interface	Formalize for benches, items, triggers	Sudhish	`feature/iinteractable`
Audio scoping	Camera-bound SFX — only play when source is within camera view	Zach	`feature/audio-scoping`
Camera refactor	Refactor camera system for cleaner follow/bounds/room handling	Zach	`feature/camera-refactor`
Custom voice lines	Record and integrate custom voice lines into audio pipeline	Zach	`feature/custom-voice-lines`
Magic numbers sweep	Extract remaining magic numbers in each owner's area to `GameConstants`	All	`feature/magic-numbers-s5`

6. Process (Niko — primary, All — code reviews)

Task	Description	Assignee	Branch
------	-------------	----------	--------

```

|-----|-----|-----|-----|
| Code reviews (readability) | Each member reviews one class for readability | All | *(PR comments)* |
| Code reviews (quality) | Each member reviews one class for code quality/maintainability | All | *(PR comments)* |
| Code metrics | Run weekly code metrics (VS Analyze / Roslyn); track in spreadsheet | Niko | `core/code-metrics` |
| README update | Controls, known bugs, tools/processes | Niko | `core/readme-update` |
| Sprint reflection | Brief burndown-based reflection | Niko | `core/sprint-reflection` |
| Branch + PR tracking | Daily check on Thomas's branches + flag blockers same-day | Niko | *(no branch)* |

```

Documentation - Did they do what they said they would do?

Yes!

Everyone was able to complete the features we wanted to have during this sprint. However, it is important to note the roles of who was implementing what switched during the sprint. Ex. Thomas worked on a boss fight with Zach and Sudhish worked on layers.

Documentation - Did they document what slipped or changed?

What Slipped:

- The boss fight implementation took so much longer than expected, therefore that definitely was a slip.
- Sri's start screen was a bit of a slip as it didn't look exactly pleasing with the hollowknight start screen, but then he was able to fix it.

What Changed:

- Roles throughout the sprint, EX. Thomas was working on layering but then switched to boss fight implementation due to thomas being interested in working on boss fight
- In general with this sprint there was a great change in how often the group met up in person. This was due to the fact there was a lot of things that needed to be implemented so the group met up much more frequently

Documentation - Version control used effectively.

Yes. Look at this branch structure:

```

## Git Branching Strategy

...

dev
├── feature/carry-overs (Sri + Thomas + Sudhish)
│   ├── feature/hud-health-fix (Sri)
│   ├── feature/ability-ui (Sri)
│   ├── feature/downslash-fix-v2 (Sri)
│   ├── feature/first-dungeon-complete (Thomas)
│   └── feature/first-dungeon-assist (Sri)

```

```

└─ feature/boss (Zach + Sudhish)
|   └─ feature/boss-behavior (Zach)
|   └─ feature/boss-room (Zach)
|   └─ feature/boss-integration (Sudhish)
└─ feature/layers (Niko + Sudhish + Thomas)
|   └─ feature/layers-visual (Niko)
|   └─ feature/layers-visual-assets (Thomas)
|   └─ feature/layers-collision (Sudhish)
└─ feature/second-dungeon (Thomas + Zach)
|   └─ feature/second-dungeon-layout (Thomas)
|   └─ feature/second-dungeon-sprites (Thomas)
|   └─ feature/second-dungeon-xml (Zach)
└─ feature/polish (Sudhish + Zach)
|   └─ feature/ibreakable (Sudhish)
|   └─ feature/iinteractable (Sudhish)
|   └─ feature/audio-scoping (Zach)
|   └─ feature/camera-refactor (Zach)
|   └─ feature/custom-voice-lines (Zach)
|   └─ feature/magic-numbers-s5 (All)
└─ core/process (Niko)
    └─ core/code-metrics (Niko)
    └─ core/readme-update (Niko)
    └─ core/sprint-reflection (Niko)
...

```

Merge Order

1. Leaf branches merge into their parent feature branch via PR
2. Parent feature branches merge into ``dev``
3. All PRs require at least one review before merging
4. Carry-over branches merge first (Week 1) so feature work builds on a clean ``dev``